

DOCUMENTACIÓN TÉCNICA ACTUALIZADA

PPM — Personal Process Manager

Sistema integral de administración de finanzas personales

Versión	1.1 — Basada en código fuente
Plataforma	Desktop — Windows / macOS / Linux (Avalonia UI)
Moneda Base	Guaraní Paraguayo (₲)
Estado	En desarrollo

SECCIÓN 01: Descripción General del Proyecto

¿Qué es PPM?

PPM (Personal Process Manager) es una aplicación de escritorio diseñada para que el usuario pueda administrar sus finanzas personales de manera integral, visualizando su situación económica en el pasado, el presente y el futuro.

El sistema gira en torno al **salario** y otros **ingresos** declarados por el usuario. A partir de estos valores se calculan automáticamente todos los indicadores financieros: saldo disponible, proyecciones de deuda, carga mensual de gastos y estado detallado de tarjetas de crédito.

Objetivos del sistema

- **1. Centralizar la información financiera:** Concentrar toda la data del usuario en un solo lugar seguro. (Alta)
- **2. Automatización del saldo:** Calcular automáticamente el saldo disponible en base al salario y otros ingresos. (Alta)
- **3. Proyección temporal:** Registrar y proyectar deudas a lo largo del tiempo de forma visual. (Alta)
- **4. Gestión avanzada de crédito:** Controlar tarjetas de crédito con historial de movimientos, cálculo de intereses y pagos mínimos. (Alta)
- **5. Control de egresos:** Registrar gastos mensuales y permitir su comparación activa con períodos anteriores. (Alta)
- **6. Monitoreo en tiempo real:** Proveer un dashboard unificado con el estado financiero consolidado. (Alta)
- **7. Dinamismo salarial:** Permitir actualizar el salario dinámicamente y reflejar el cambio en cascada de forma automática. (Media)
- **8. Portabilidad de datos:** Exportar resúmenes financieros estructurados en archivos de texto plano. (Media)

Alcance

La aplicación corre de forma **100% local**, sin dependencias externas de conexión a internet. Los datos se almacenan de manera persistente en una base de datos SQLite en el equipo del usuario. La aplicación garantiza compatibilidad nativa con Windows, macOS y Linux gracias a su desarrollo sobre Avalonia UI.

NOTA DE CONFIGURACIÓN MONETARIA

La moneda base del sistema es el Guaraní Paraguayo (₲). Todos los campos numéricos se autoformatean con puntos de miles integrados y no admiten valores decimales bajo ninguna regla del negocio.

SECCIÓN 02: Stack Tecnológico

El núcleo técnico se sustenta sobre arquitecturas robustas estandarizadas para el ecosistema corporativo local.

Componentes Backend

Componente	Tecnología	Detalle Técnico
Lenguaje	C#	Entorno de ejecución moderno sobre .NET 8
ORM	Entity Framework Core	Versión 8.0.x para mapeo relacional
Base de datos	SQLite	Archivo estructurado local .db sin motor externo
Encriptación	BCrypt.Net-Next	Paquete NuGet avanzado con adición automática de Hash + Salt
Arquitectura	Clean Architecture	Desacoplamiento estricto por capas: Domain / Application / Infrastructure / UI
Patrón de datos	Repository Pattern	Interfaces fuertemente tipadas y definidas en la capa Domain
Inyección de dep.	MS DependencyInjection	Contenedor nativo integrado por defecto en .NET 8

Componentes Frontend

Componente	Tecnología	Detalle Técnico
Framework UI	Avalonia UI	Renderizado nativo multiplataforma (Windows / macOS / Linux)
Patrón UI	MVVM	Patrón estricto Model-View-ViewModel para separar lógica e interfaz
Binding toolkit	CommunityToolkit.Mvvm	Uso nativo de atributos de compilación <code>ObservableProperty</code> y <code>RelayCommand</code>
Estilos	Avalonia Styles	Diseño UI minimalista y limpio personalizado en diccionarios XAML

Arquitectura de capas

El proyecto se organiza bajo los principios estrictos de Clean Architecture, garantizando que ninguna capa interna posea conocimientos de capas externas:

- **PPM.Domain:** Núcleo puro del negocio. Contiene Entidades, enumeradores (`IncomeType` , `ExpenseType` , etc.) e interfaces de repositorio primarias. Cero dependencias externas.
- **PPM.Application:** Casos de uso de la aplicación y la orquestación de flujos. Conoce únicamente la capa Domain.
- **PPM.Infrastructure:** Implementación de repositorios de datos y el `DbContext` de persistencia usando EF Core + SQLite.
- **PPM.UI:** Vistas estructuradas en Avalonia, ViewModels reactivos (MVVM) y la configuración del inyector de dependencias global.

SECCIÓN 03: Módulos Funcionales

3.1 Autenticación y Registro (Users)

El sistema cuenta con una pantalla segura de inicio de sesión y registro de credenciales.

- Validaciones lógicas directas contra la base de datos local (tabla `User`).

- Contraseñas protegidas mediante algoritmos asimétricos Bcrypt (nunca almacenadas en texto plano).

3.2 Módulo de Salarios e Ingresos Adicionales (Salaries / Incomes)

- **Salario:** El usuario declara su salario mensual neto bajo la entidad salary . La plataforma almacena y grafica de forma inmutable el histórico completo de cambios salariales.
- **Ingresos Adicionales:** Registro ágil de entradas extraordinarias (entidad Income) categorizadas explícitamente por tipo: Bono , Freelance , Regalo , Venta u Otros .
- Los estados del balance financiero global se recalculan integrando en tiempo real la sumatoria del salario activo más los ingresos devengados en el mes en curso.

3.3 Módulo de Deudas (Debts)

Permite registrar compromisos financieros fijos a mediano y largo plazo, tales como préstamos bancarios o compras financiadas en cuotas.

- **Campos del sistema:** Nombre identificativo, Descripción, Monto Total, Plazo estipulado en meses y Estado actual (Pending , Paid).
- **Procesamiento automatizado:** Cálculo matemático de la cuota mensual estimada, totales acumulados y balance neto de deuda restante.

3.4 Módulo de Tarjetas de Crédito (CreditCards / CardMovements)

Seguimiento avanzado de líneas de crédito activas.

- **Campos de tarjeta:** Nombre de la entidad, Límite de crédito asignado, Tasa de interés efectiva y Fechas fijadas de cierre/vencimiento.
- **Movimientos de control (CardMovement):** Historial detallado por cada tipo de transacción (Consumo O Pago).
- **Automatización de intereses:** El interés compuesto generado al corte se asienta de manera automática como un consumo bajo el concepto descriptivo "Interés de cierre", manteniendo la integridad exacta del saldo cuadrado.

3.5 Módulo de Gastos Mensuales (Expenses)

Permite mapear con precisión quirúrgica las salidas de dinero cotidianas del mes.

- **Estructura:** Descripción detallada, Monto y Tipo de gasto clasificado semánticamente (Housing , Food , Transport , Entertainment , Health , Other).

- **Lógica analítica:** Totalización instantánea de consumos del mes activo y métricas comparativas directas contra el periodo mensual inmediato anterior.

3.6 Dashboard Principal

Consolidación y visualización inmediata de datos económicos críticos para el usuario a través de una sola vista fluida:

- Saldo global disponible y remanente estimado.
- Mapeo de deudas activas junto a su impacto en la carga financiera mensual.
- Resumen analítico ejecutivo de tarjetas vigentes y distribución de gastos diarios.

SECCIÓN 04: Lineamientos de UI / UX

La interfaz sigue rigurosamente principios de diseño minimalista contemporáneo inspirados directamente en las *Apple Human Interface Guidelines*.

- **Jerarquía visual:** Establecimiento estricto de pesos y contrastes tipográficos para guiar el flujo ocular del usuario sin saturación de componentes.
- **Feedback inmediato:** Respuestas visuales interactivas instantáneas en controles de entrada de datos y transiciones de carga de registros.
- **Formato de moneda nativo:** Inyección forzada de separadores de miles tipográficos mediante punto (ej. 1.500.000 ₡). Ausencia total de decimales. El símbolo nativo (₡) opera de manera estricta como prefijo o sufijo según el espacio visual disponible.

SECCIÓN 05: Modelo de Base de Datos

Las tablas y entidades del esquema de persistencia se modelan exclusivamente en inglés, siguiendo los estándares internacionales de buenas prácticas en ingeniería de software.

Entidad (C#)	Mapeo / Función BD	Descripción del Dominio
User	Usuarios	Datos de inicio de sesión de cuentas (username y hash seguro de contraseña).
Salary	Salarios	Historial inmutable de modificaciones de salarios mensuales declarados.
Income	Ingresos Extras	Estructura de ingresos adicionales categorizados mediante el enum <code>IncomeType</code> .
Debt	Deudas	Registro de compromisos crediticios a plazos bajo bandera de control <code>DebtStatus</code> .
CreditCard	Tarjetas	Esquema maestro de límites de crédito, tasas contractuales y fechas operativas de corte.
CardMovement	Libro de tarjeta	Libro contable auxiliar de transacciones detalladas de consumos y abonos (<code>CardMovementType</code>).
CreditCardPayment	Pagos de tarjeta	Historial histórico unificado de amortizaciones y pagos consolidados a plásticos de crédito.
Expense	Gastos	Registro y clasificación de salidas monetarias ligadas al enumerador estricto <code>ExpenseType</code> .

LÍNEA DE COMANDOS DE MIGRACIÓN ACTIVA

Para actualizar de manera consistente el esquema relacional local, ejecute el siguiente comando en la consola de desarrollo:

```
dotnet ef migrations add InitialCreate --project src/PPM.Infrastructure --startup-project src/PPM.UI
```

SECCIÓN 06: Requerimientos del Proyecto Académico

Mapeo detallado del cumplimiento de las directrices de la Universidad Autónoma de Asunción (UAA) dentro de la arquitectura de la solución PPM.

Requerimiento UAA	Implementación Real y Cobertura en PPM-UAA
3 operaciones/consultas complejas	Procesamiento integrado del Dashboard, lógicas de proyección predictiva cruzada (Incomes vs Expenses) y reportes automatizados.
6 CRUDs completos independientes	Módulos totalmente operativos con persistencia completa implementados para: Users , Salaries , Debts , CreditCards , Incomes y Expenses .
Lectura / escritura de archivos externos	Módulo central en desarrollo enfocado en la exportación estructurada de resúmenes financieros mensuales a ficheros físicos de texto legible.
Menú principal modular	Interfaz moderna responsiva basada en Avalonia UI estructurada con un Sidebar de navegación fluida por vistas lógicas desacopladas.
Login seguro y encriptado	Mecanismo de seguridad nativo implementado en la infraestructura mediante hashing criptográfico con BCrypt.Net-Next .
Uso de Tipos Enumerados (Enums)	Estructuración y tipado fuerte a través de Enums clave: ExpenseType , DebtStatus , UserRole , IncomeType y CardMovementType .
Gestión robusta de Excepciones	Bloques de control estructurados try/catch/finally integrados de manera nativa en capas de repositorios de datos y ViewModels de interfaz.
Controles UI responsivos	Diseño de maquetación elástica mediante el uso avanzado de controles de distribución Grid , StackPanel y Data Binding reactivo bidireccional en archivos descriptivos .axaml .

SECCIÓN 07: Entregables del Proyecto

La estructura de empaquetado final para la entrega académica se organizará estrictamente bajo la siguiente jerarquía de archivos dentro del contenedor comprimido de la raíz:

1. **Código Fuente Completo:** Inclusión total de la solución global estructurada en el archivo `.sln` junto a los subproyectos desacoplados mapeados en el directorio relativo `/src`.
2. **Base de Datos Autoejecutable:** Al iniciar el despliegue inicial de la solución, el ecosistema de persistencia `ppm.db` se autogenerará dinámicamente en el directorio local de compilación final gracias a la ejecución nativa de los scripts de migración de Entity Framework Core.
3. **Documentación Técnica de Soporte:** Inclusión íntegra del presente documento técnico de ingeniería en su versión actualizada y revisada.